

Pilotez vos agents de code

La méthode complète pour déléguer des missions à un agent de code — sans perdre le contrôle du repo, du budget ni de la qualité.

> Promesse mesurable : en 2 heures de travail, vous déléguez une mission complète à un agent de code — de la rédaction de la fiche de tâche au merge validé — sans perdre le contrôle. Vous savez choisir votre agent, cadrer la mission, rédiger le prompt, relire le diff, imposer les tests et orchestrer plusieurs agents.

Cible, prérequis, durée

|||

|---|---|

| Cible | Développeurs en poste, alternants et juniors, solopreneurs tech |

| Prérequis | Savoir lire du code, lancer des commandes dans un terminal, comprendre ce qu'est un test. Pas besoin d'être senior. |

| Durée | 2 h de lecture active (modules 1 à 3) + 1 semaine de pratique (modules 4 à 6) |

| Matériel | Un terminal, git, un compte sur un fournisseur d'API ou un agent installé, un repo d'entraînement |

Le contexte : pourquoi cette formation existe

Les coding agents ont envahi le marché. Les dépôts open source les plus étoilés de 2026 ne sont plus des frameworks web : ce sont des harness d'agents de code — deepseek-harness (110 000 étoiles), CodexPlusPlus (28 000 étoiles) et leurs cousins. Le signal est clair : le marché a validé « rendre les agents de code utilisables ».

Le problème ? Personne n'enseigne comment les piloter. On trouve des tutos « installez l'agent X », des démos « regardez ce qu'il sait faire », mais presque rien sur la seule compétence qui compte vraiment :

> Savoir transformer une intention floue en mission cadrée, et garder la main sur ce que l'agent produit.

Un agent de code est un collaborateur brillant, rapide, infatigable — et totalement dépourvu de jugement. Il ne sait pas ce que vous vouliez dire, il ne sait pas ce qui est hors périmètre, il ne sait pas ce que « fini » veut dire. Si vous ne le pilotez pas, il invente. Si vous le pilotez bien, il vous fait gagner des heures par jour.

Cette formation vous donne la méthode. Pas une liste de prompts magiques : un système de pilotage, module après module, avec des commandes exactes, des exemples réels et des exercices à faire sur

vos propres repos.

Comment utiliser cette formation

1. Modules 1 à 3, en une soirée : vous construisez votre socle — choisir l'agent, préparer le repo, rédiger la mission. À la fin du module 3, vous lancez votre première mission réelle.
2. Modules 4 à 6, sur vos missions : vous ne les lisez pas d'un bloc — vous les consultez quand le besoin arrive (review, tests, orchestration).
3. Les templates : les 5 fichiers de templates/ accompagnent chaque module. Copiez-les dans votre repo de travail, remplissez-les, utilisez-les. C'est le pack complet.
4. La checklist finale : une fois tout lu, faites le point en 10 minutes. Ce qui n'est pas coché, vous savez où le trouver.

Chaque module suit la même structure : la leçon (dense, actionnable), l'exemple réel (une situation concrète, de bout en bout), l'exercice (à faire, avec critères de réussite vérifiables).

Module 1 — Choisir son agent (et son modèle)

La leçon

1.1 Le paysage en 2026 : quatre familles

Tous les « agents de code » ne se ressemblent pas. Quatre familles coexistent, et le choix de la famille est plus important que le choix de la marque.

Famille A — Les agents en ligne de commande (CLI).

Claude Code, Codex CLI, Gemini CLI, Aider, et les harness open source (deepseek-harness, CodexPlusPlus). Vous les lancez dans votre terminal, dans votre repo. Ils lisent les fichiers, éditent, exécutent des commandes, proposent un plan avant d'agir.

- Points forts : contrôle maximal (approbation par étape), transparence totale (vous voyez chaque commande), coût maîtrisé, s'intègre à votre git.
- Points faibles : pas d'interface visuelle, courbe d'apprentissage du pilotage.
- Pour qui : tout le monde, en priorité. C'est la famille de référence de cette formation.

Famille B — Les agents intégrés à l'IDE.

Cursor, Windsurf, GitHub Copilot (mode agent), JetBrains AI. L'agent vit dans votre éditeur, voit votre sélection, propose des edits en diff visuel.

- Points forts : confort maximal, contexte visuel (vous montrez le fichier), onboarding doux.
- Points faibles : contrôlabilité variable, coût parfois élevé (abonnement + tokens), tendance à l'édition « magique » difficile à auditer.
- Pour qui : l'exploration, le refactoring guidé, ceux qui vivent dans l'éditeur.

Famille C — Les agents autonomes (asynchrones).

OpenHands, Devin et les agents cloud des plateformes (Codex cloud, Claude Code cloud). Vous donnez une mission, l'agent travaille dans son propre environnement, parfois pendant des heures, et revient avec un résultat.

- Points forts : délégation complète, idéal pour les missions longues et indépendantes.
- Points faibles : contrôle différé, environnement sandbox distinct du vôtre, coût à surveiller, review obligatoire a posteriori.
- Pour qui : les missions « lancer et revenir plus tard » (migration, génération de tests, scripts).

Famille D — Les agents que vous construisez (frameworks).

LangGraph, CrewAI, ou vos propres boucles via API. Vous orchestrez vous-même modèles, outils et mémoire.

- Points forts : flexibilité absolue, coût optimisé, pas de dépendance à un éditeur.
- Points faibles : tout est à construire, maintenance lourde.
- Pour qui : les solopreneurs qui industrialisent (cette formation vous donne les bases au module 6).

1.2 Les cinq critères de choix

Quand vous comparez deux agents, posez ces cinq questions dans l'ordre. Si un agent échoue sur le premier critère, inutile de regarder les suivants.

Critère 1 — Contrôlabilité : puis-je approuver avant qu'il agisse ?

Le minimum vital : l'agent affiche un plan et demande l'accord avant d'éditer des fichiers ou d'exécuter des commandes. Si l'agent agit seul et que vous découvrez le résultat après coup, vous n'êtes pas en train de piloter — vous subissez. Modes d'approbation à chercher : « demander à chaque étape », « demander pour les commandes », « plan à valider en début de mission ».

Critère 2 — Transparence : puis-je voir chaque action ?

Chaque commande exécutée, chaque fichier modifié, chaque diff : tout doit être visible et journalisé. L'agent qui « fait des choses » sans que vous puissiez reconstituer le fil exact est un risque. Vérifiez : journal des commandes, liste des fichiers modifiés, diff avant/après.

Critère 3 — Coût réel par mission : que vais-je payer ?

Le coût d'un agent de code se mesure en tokens consommés, pas en euros d'abonnement. Une mission typique « ajouter une fonctionnalité avec tests » consomme en moyenne :

- Entrée (le contexte, les fichiers lus, l'historique) : 150 000 à 400 000 tokens
- Sortie (le code produit) : 10 000 à 30 000 tokens

À titre indicatif, avec un modèle courant à ~3 \$/M tokens en entrée et ~15 \$/M en sortie (tarifs publics 2026, ils baissent régulièrement), une mission coûte 1 à 4 €. Avec des modèles low-cost (DeepSeek, Llama via API ou local), le même travail descend sous 0,50 €. Le vrai coût n'est pas le token : c'est l'heure perdue quand la mission est mal cadrée (voir module 3).

Critère 4 — Fenêtre de contexte : quelle quantité de code voit-il ?

La fenêtre de contexte, c'est la « mémoire de travail » de l'agent. Une mission sur un gros repo exige de l'agent qu'il lise l'architecture, les conventions, les fichiers voisins. Plus la fenêtre est large, moins vous

prenez de temps à lui indiquer les fichiers à lire. En pratique : 200k tokens et plus, c'est confortable ; en dessous, compensez par un AGENTS.md précis (module 2) et un périmètre serré (module 3).

Critère 5 — Écosystème : avec quoi s'intègre-t-il ?

MCP (Model Context Protocol), hooks, intégration git, support de vos langages, mode headless pour la CI. Un agent qui accepte les serveurs MCP (accès à votre base, à votre outil de ticketing...) se branche sur votre travail réel au lieu de travailler à côté.

1.3 Le choix selon votre profil

- Dev en poste, équipe établie : CLI (famille A) pour les missions cadrées + agent IDE (famille B) pour l'exploration. Vous gardez la review humaine comme garde-fou — c'est votre métier.
- Alternant / junior : commencez par un CLI (famille A) sur un repo d'entraînement. L'objectif n'est pas la vitesse, c'est la compréhension : voir chaque diff, comprendre chaque décision, apprendre en relisant.
- Solopreneur tech : CLI pour le quotidien (famille A), autonome pour les missions longues (famille C), et vous monterez votre orchestration (famille D) après le module 6.

1.4 Le piège n°1 : choisir sur la hype

Le meilleur agent du moment n'est pas le plus étoilé, ni celui de la démo la plus impressionnante. C'est celui que vous arriverez à piloter dans votre workflow. La règle : une seule famille à la fois. Installez un CLI, faites les exercices des modules 2 et 3 avec lui, et ce n'est qu'après trois missions réussies que vous comparez avec un autre.

L'exemple réel

Situation. Mégane, 22 ans, alternante dans une PME. Son tuteur lui donne une mission : « ajouter un export CSV des commandes ». Elle n'a jamais utilisé d'agent de code. Elle hésite entre trois options vues sur les réseaux : l'agent autonome à la mode, l'extension de son IDE, et le CLI que son tuteur utilise.

Ce qu'elle fait (application de la grille).

1. Contrôlabilité : l'agent autonome travaille dans son propre sandbox — elle ne verrait le résultat qu'à la fin. Éliminé pour une première mission.
2. Transparence : l'extension IDE propose des edits visuels — confortable, mais elle ne voit pas les commandes exécutées (tests lancés ? migration appliquée ?). Réservé à plus tard.
3. Coût : le CLI de son tuteur consomme ~200k tokens en entrée pour ce type de mission → moins de 2 €. Acceptable.
4. Contexte : fenêtre large, et le repo de la PME est petit. OK.
5. Écosystème : le CLI est déjà configuré sur le repo de l'entreprise, avec un AGENTS.md en place.

Résultat. Mégane choisit le CLI de son tuteur. Elle fait l'exercice ci-dessous, puis sa première mission — et elle peut montrer à son tuteur un journal complet : plan validé, fichiers modifiés, tests passés. La confiance s'installe en une démonstration, parce que tout est traçable.

L'exercice — Module 1

Objectif : choisir votre agent sur des critères, pas sur la hype.

1. Listez 3 agents (ou familles) que vous envisagez.
2. Pour chacun, notez de 1 à 5 les cinq critères de la section 1.2, avec une justification d'une ligne.
3. Éliminez tout agent noté 1 ou 2 en contrôlabilité.
4. Choisissez le gagnant, installez-le sur votre machine, et lancez `--help` (ou l'équivalent) pour vérifier qu'il affiche bien les options d'approbation et de plan.

Critères de réussite :

- ☐ Vous avez éliminé au moins un agent sur le critère de contrôlabilité
- ☐ Vous savez expliquer en une phrase le coût estimé d'une mission type sur votre agent
- ☐ Vous avez identifié dans la doc de votre agent les modes : « plan », « approbation », « budget »

Module 2 — Setup d'un repo de travail

La leçon

2.1 Le principe : un repo est un contrat

Un agent de code ne comprend votre projet que par ce qu'il peut lire. S'il doit deviner la stack, les commandes, les conventions et les pièges, il va deviner — mal. Le module 2 consiste à rendre votre repo lisible par un agent : un fichier de contexte, des branches propres, des limites claires, un environnement reproductible.

2.2 Le fichier de contexte : AGENTS.md

La plupart des agents lisent automatiquement un fichier de contexte à la racine du repo. Le standard émergent s'appelle AGENTS.md (accepté par les principaux CLI et IDE ; certains lisent aussi CLAUDE.md ou .cursorrules — le plus sûr est AGENTS.md, parfois complété par un alias). Ce fichier est votre contrat permanent avec l'agent : tout ce que vous voulez qu'il sache à chaque mission.

Ce qu'on y met (et rien d'autre) :

1. La stack, en une section : langages, frameworks, versions, gestionnaire de paquets. L'agent n'a pas à découvrir que le projet est en TypeScript strict avec pnpm.
2. Les commandes canoniques : install, test, lint, build, dev. Une seule ligne chacune. L'agent les exécute au lieu d'inventer.
3. L'architecture en 10 lignes : où vivent les routes, les modèles, les services, les tests. Assez pour qu'il ne cherche pas où ranger son code.
4. Les conventions : style, nommage, gestion des erreurs, i18n, type. Ce que votre revue de code exigerait de tout développeur.

5. Les pièges connus : ce qui a déjà cassé (migration qui traîne, dépendance à un service local, tests lents). L'agent ne répétera pas les erreurs que l'équipe a payées.
6. Les règles absolues : « jamais de secret en dur », « jamais de modification des tests sans justification », « toujours exécuter les tests avant de terminer ». Des impératifs courts, non négociables.

2.3 Git hygiene : la base du contrôle

- Une branche par mission : feature/<identifiant>-<slug> (ex. feature/PAIEMENT-STRIFE). L'agent travaille sur cette branche ; main reste intouchable. C'est votre filet : quoi qu'il fasse, main est saine.
- Jamais de push direct sur main : même en solo, protégez la branche (protection GitLab/GitHub, ou simplement la discipline de toujours passer par une branche).
- Commits atomiques et conventionnels : un commit = une étape logique, message en conventional commits (feat:, fix:, test:, refactor:). Cela rend le diff lisible et l'historique reconstituable — indispensable quand c'est l'agent qui commit.
- Vérifiez ce que l'agent a commité : le module 4 vous donne la méthode, mais dès le setup, décidez : l'agent a-t-il le droit de committer lui-même ? Recommandation : oui, mais jamais de push — vous poussez après review.

2.4 Le budget : le second filet

Les agents ont presque tous des limites configurables. Activez-les toutes :

- Budget de temps : 15-30 minutes par mission courte, 2 h par mission longue. Passé ce délai, l'agent s'arrête et rend compte.
- Budget de tokens / de coût : fixez un plafond (ex. 500k tokens d'entrée). L'agent s'arrête proprement s'il le dépasse.
- Permissions de commandes : whitelist (npm test, npm run lint, git add, git commit...) et blacklist (rm -rf, git push, npm publish, modifications de package.json hors ajout de dépendance explicitement autorisé).
- Permissions de fichiers : autorisez les zones du périmètre, interdisez le reste (voir module 3).

2.5 L'environnement reproductible

Un agent qui ne peut pas lancer les tests parce que « ça marche chez moi » est inutilisable. Exigez :

- Un README.md d'installation en 3 commandes max.
- Un Makefile (ou justfile) avec les cibles canoniques : make install, make test, make lint, make dev.
- Des lockfiles commités (package-lock.json, pnpm-lock.yaml, uv.lock...).
- Des tests qui tournent en moins de 2 minutes (les agents les relancent souvent ; des tests lents = des missions lentes).

2.6 Le piège n°2 : l'agent qui s'égare

Sans AGENTS.md, l'agent explore le repo, devine la stack, exécute des commandes au hasard, et finit par modifier des fichiers hors de la mission « parce que c'était lié ». Le setup n'empêche pas tout — mais il transforme l'égarement en exception, pas en règle. Et quand l'agent s'égare quand même, le périmètre du module 3 et la review du module 4 le rattrapent.

L'exemple réel

Situation. Karim, solopreneur, vend un outil SaaS en Next.js. Il veut déléguer à un agent. Il passe 40 minutes à préparer son repo une bonne fois pour toutes.

Ce qu'il fait.

1. Il crée un AGENTS.md de 45 lignes : stack (Next 15, TypeScript strict, Tailwind, Prisma/Postgres), commandes (pnpm install, pnpm test, pnpm lint, pnpm build), architecture (routes dans app/, logique métier dans lib/, DB via Prisma), conventions (composants client/server marqués, erreurs en français côté UI), pièges (migrations Prisma à générer et appliquer en deux étapes, tests e2e qui exigent Postgres local), règles absolues (pas de secret en dur, pas de any, tous les tests verts avant de terminer).
2. Il crée le Makefile qui pointe vers ces commandes.
3. Il configure les limites de son CLI : approbation à chaque étape, budget 15 min pour les missions courtes, whitelist de commandes, interdiction de git push.
4. Il teste : il demande à l'agent « résume ce projet en 10 lignes ». L'agent cite le contenu d'AGENTS.md presque mot pour mot. Le contrat est lu.

Résultat. Les missions suivantes démarrent avec 80 % du contexte déjà fourni. Karim ne répète plus « attention, c'est du Prisma, utilise pnpm » dans chaque prompt — c'est écrit une fois, lu à chaque fois.

L'exercice — Module 2

Objectif : rendre votre repo (ou un repo d'entraînement) pilotable par un agent.

1. Créez ou complétez AGENTS.md avec les 6 sections de la leçon (stack, commandes, architecture, conventions, pièges, règles absolues). Tenez-le à 60 lignes maximum.
2. Créez Makefile avec les cibles install, test, lint, build, dev.
3. Créez une branche feature/PREMIERE-MISSION et vérifiez que main est protégée (ou que vous vous interdisez de pousser dessus).
4. Configurez sur votre agent : approbation par étape, budget de temps, whitelist de commandes, interdiction de push.
5. Test de contrat : demandez à l'agent de résumer le projet en 10 lignes.

Critères de réussite :

- [] Le résumé de l'agent reprend au moins 3 informations précises de votre AGENTS.md (pas des généralités)
- [] L'agent exécute make test (ou équivalent) sans que vous lui donniez la commande
- [] L'agent refuse (ou demande confirmation) quand vous lui demandez de pousser sur main
- [] Votre AGENTS.md tient en 60 lignes

Module 3 — Le prompt de mission parfait

La leçon

3.1 Le changement de paradigme : le prompt est un document

La plupart des gens écrivent des prompts comme des messages de chat : « bonjour, tu peux m'ajouter un truc pour payer en ligne ? » L'agent répond, on discute, on précise, on repart... 40 minutes plus tard, le résultat est approximatif et personne ne sait qui a décidé quoi.

La méthode de cette formation : le prompt de mission est un document de travail, rédigé avant d'ouvrir l'agent, au même titre qu'une fiche de tâche. On le relit comme un ticket, on le corrige avant de l'envoyer, et l'agent l'exécute de bout en bout.

3.2 L'anatomie en 7 blocs

Bloc 1 — Rôle & contexte.

« Tu es développeur senior sur le repo X. Stack : ... Tu travailles sur la branche feature/... déjà créée. » Donnez l'identité, le repo, la branche, les contraintes de base. Un agent qui ne sait pas sur quel repo il est travaille au hasard.

Bloc 2 — Objectif mesurable.

Pas « ajoute le paiement », mais : « un client peut payer une commande par carte ; le parcours complet passe les 5 tests d'intégration de tests/paiement.test.js ». Un objectif est mesurable quand vous pouvez dire, sans ambiguïté, s'il est atteint. Le meilleur objectif mesurable cite un test, une commande ou un critère observable.

Bloc 3 — Périmètre.

Deux listes : autorisé (fichiers et zones où l'agent peut écrire) et interdit (fichiers qu'il ne doit pas toucher, commandes qu'il ne doit pas lancer). Le périmètre est la première ligne de défense contre l'égarement. Soyez précis : « src/paiement/, db/migrations/, tests/paiement.test.js — interdit : src/front/, package.json (sauf ajout de la dépendance stripe), modifier les tests existants ».

Bloc 4 — Contraintes techniques.

Style, performances, sécurité, accessibilité, i18n : tout ce que votre revue de code aurait exigé. « Clé API lue depuis process.env (jamais en dur) », « async/await, pas de callbacks », « réponses JSON {error: "..."} », « composants accessibles (aria) ».

Bloc 5 — Définition de fini (Definition of Done).

La liste des conditions pour que la mission soit considérée comme terminée. C'est le bloc qui change tout : sans lui, l'agent s'arrête quand il est « content de son code » — avec lui, il s'arrête quand les tests passent, le lint est propre, les types sont vérifiés et la doc est à jour.

Bloc 6 — Livrables.

Ce que l'agent doit produire en plus du code : commits atomiques en conventional commits, résumé des changements, liste des risques, éventuellement un plan de test manuel. Un agent qui livre « du code » sans résumé vous oblige à tout relire ; un agent qui livre « un diff + un résumé + les risques » se fait reviewer en 10 minutes.

Bloc 7 — Format de sortie.

« Réponse finale en français, max 250 mots : ce qui a été fait, ce qui n'a pas été fait, les risques restants. » Cadrez la langue, la longueur et la structure. Cela rend la review rapide et comparable d'une mission à l'autre.

3.3 Le pattern spec-first

Avant d'écrire le prompt, écrivez la fiche de tâche (template spec-fiche-tache.md) : identifiant, titre, objectif, contexte, périmètre, contraintes, définition de fini, livrables, dépendances, estimation. La fiche est votre réflexion ; le prompt est son expression. Si vous ne pouvez pas remplir la fiche, la mission n'est pas mûre — et l'agent ne la rendra pas meilleure.

Pour les missions complexes, découpez : une mission = une fiche = un prompt. « Ajoute le paiement Stripe » n'est pas une mission, c'est un projet. « Ajoute le endpoint de création d'intention de paiement » en est une. Le module 6 détaille le découpage.

3.4 Les six erreurs qui coûtent des heures

1. Objectif vague (« améliore ce code ») → l'agent « améliore » au hasard, souvent en réécrivant ce qui marchait.
2. Pas de périmètre → l'agent refactorise les fichiers voisins « pour la cohérence ».
3. Pas de définition de fini → l'agent s'arrête à « ça compile », les tests échouent, il « corrige » les tests au lieu du code.
4. Plusieurs missions dans un seul prompt → impossible à valider, impossible à relire, l'agent priorise à sa façon.
5. Prompt sans contexte repo → l'agent réinvente les conventions ; le résultat ne ressemble à rien du projet.
6. Pas de format de sortie → réponse fleuve de 2 000 mots, ou pire : l'agent s'étend au-delà de la mission.

3.5 La boucle de feedback en 3 temps

Quand le résultat n'est pas bon, ne partez pas dans un ping-pong de 15 messages. Structurez votre retour en trois temps, dans l'ordre :

1. Ce qui est bon (2-3 points) — l'agent conserve ce qui fonctionne.
2. Ce qui manque (par rapport à la définition de fini, jamais par rapport à un goût) — citez la ligne du bloc 5.
3. Ce qui est à refaire (précisément, fichier par fichier) — « le endpoint retourne 500 sur payload vide ; corrige et ajoute un test pour ce cas ».

Un feedback structuré se traite en un aller-retour. Un « ça va pas, recommence » se traite en dix.

3.6 Le piège n°3 : le prompt parfait qui masque une mission pourrie

Le plus beau prompt du monde ne sauve pas une mission mal pensée. Si l'objectif est flou dans votre tête, il sera flou dans le prompt. La règle : si vous ne pouvez pas écrire la fiche de tâche en 10 minutes,

la mission n'est pas prête. Découpez, précisez, ou délégez autre chose.

L'exemple réel

Situation. Karim (du module 2) veut le paiement Stripe dans son SaaS. Il rédige la fiche de tâche (10 min), puis le prompt de mission avec les 7 blocs.

Version ratée (ce que font 90 % des gens) :

> « Salut, ajoute le paiement Stripe au projet. Il faut un truc propre. Merci. »

Ce que l'agent fait : il choisit la stack de paiement à sa façon, touche 30 fichiers, ajoute des dépendances non demandées, « nettoie » le code au passage, et s'arrête quand ça compile — sans test, sans migration propre, sans résumé. Review impossible, rollback pénible.

Version pilotée (le prompt complet est dans la landing page et le template prompt-mission.md) :

> « Rôle : développeur senior sur le repo caissevite (Node 20, Express, SQLite). Branche feature/paiement-stripe déjà créée.

> Objectif : un client peut payer une commande par carte ; le parcours complet passe les 5 tests d'intégration de tests/paiement.test.js.

> Périmètre autorisé : src/paiement/, src/routes/, db/migrations/, tests/paiement.test.js. Interdit : src/front/, package.json (sauf ajout de stripe), modifier les tests existants.

> Contraintes : clé API depuis process.env.STRIPESECRETKEY, jamais en dur ; erreurs en JSON {error: "..."} ; async/await.

> Définition de fini : 5 tests verts · npm run lint · npm run typecheck · migration appliquée · aucun test existant modifié · résumé des risques.

> Livrables : commits atomiques en conventional commits, diff résumé à relire.

> Format : réponse finale en français, 250 mots max. »

Résultat. L'agent produit exactement le périmètre demandé. La review (module 4) prend 10 minutes au lieu d'une heure. Les tests existants n'ont pas bougé. Le coût total : ~2 € de tokens, 25 minutes de travail humain (fiche + prompt + review).

L'exercice — Module 3

Objectif : rédiger votre premier prompt de mission complet.

1. Prenez une mission réelle, petite (1 à 2 h de travail humain).
2. Remplissez la fiche de tâche (templates/spec-fiche-tache.md).
3. Rédigez le prompt avec les 7 blocs (templates/prompt-mission.md).
4. Relisez le prompt en vous posant les 6 questions des erreurs (3.4) — corrigez.
5. Lancez la mission, puis faites la review du module 4.

Critères de réussite :

- [] L'objectif contient un critère observable (test, commande, comportement précis)
- [] Le périmètre interdit existe et cite des fichiers précis
- [] La définition de fini contient au moins 4 conditions vérifiables
- [] L'agent n'a touché aucun fichier hors périmètre
- [] L'agent s'est arrêté en respectant votre format de sortie

Module 4 — Review des changements

La leçon

4.1 Le principe : l'agent propose, vous disposez

Un agent de code n'a pas de jugement — il a une probabilité. Sa sortie est une proposition, pas une décision. Le merge n'est jamais automatique : c'est vous qui validez, après avoir lu. Cette règle est le cœur du « sans perdre le contrôle » de la promesse de cette formation.

4.2 Lire un diff efficacement

Vous n'avez pas besoin de relire chaque fichier ligne à ligne. La méthode :

1. Vue d'ensemble : `git diff main...HEAD --stat` (ou votre branche de référence) → combien de fichiers, lesquels, combien de lignes. Un signal d'alarme immédiat : des fichiers hors périmètre, ou un nombre de lignes disproportionné par rapport à la mission.
2. Lecture ciblée : `git diff main...HEAD` puis concentrez-vous sur les zones à risque (voir 4.3).
3. Vérification des fichiers de test : `git diff main...HEAD -- '/test' '/spec'` séparément — les modifications de tests sont le point le plus dangereux d'une review d'agent (voir module 5).
4. Vérification des dépendances : regardez précisément `package.json`, `pyproject.toml`, etc. Toute dépendance ajoutée doit avoir une justification dans le résumé de l'agent.

4.3 Les 7 signatures du code d'agent

Entraînez-vous à les repérer — c'est la compétence clé du module :

1. Sur-ingénierie : abstraction inutile (« pour être propre »), factory, interface ou configuration qui n'a pas de second usage. L'agent a tendance à généraliser à partir d'un seul cas.
2. Hallucination d'API : appel à une fonction, une option ou un paramètre qui n'existe pas, ou avec une signature erronée. Détection : la compilation, les types, et les tests. Un agent qui « connaît » une API sans la vérifier est un risque classique.
3. Tests vidés ou supprimés : assertion supprimée, test remplacé par un `expect(true)`, test modifié pour passer sans vérifier le comportement. C'est la signature la plus grave (module 5).
4. Refactorisation hors périmètre : « au passage, j'ai renommé les fonctions du module voisin ». Interdit — sauf si le périmètre l'autorisait explicitement.
5. Dépendances gratuites : une lib ajoutée pour 3 lignes de code, un utilitaire « pratique » qui n'est

utilisé nulle part. Chaque dépendance est une surface d'attaque et une dette.

6. Code mort ou copié : fonctions dupliquées, imports inutilisés, blocs commentés, morceaux visibles de copier-coller d'ailleurs.
7. Commentaires bavards : // Cette fonction ajoute un élément à la liste — un commentaire qui décrit le code au lieu d'expliquer le pourquoi. Symptôme de code écrit « pour faire joli » plutôt que pensé.

4.4 Les contrôles de sécurité non négociables

- Secrets : `grep -r "sklive\|APIKEY\|password"` sur le diff — une clé commitée, même dans un commit d'essai, est une fuite à faire tourner, pas à supprimer.
- Injection : toute concaténation dans une requête SQL, une commande shell, un HTML (innerHTML) — l'agent a tendance à écrire du SQL interpolé « simple ».
- Dépendances : vérifiez la réputation d'une nouvelle dépendance avant d'accepter le merge (npm audit / pip-audit, et un coup d'œil au repo de la lib).
- Permissions : l'agent a-t-il ouvert un endpoint, un fichier, une route avec des droits plus larges que nécessaire ?

4.5 Le workflow de validation en 5 étapes

1. Lire le résumé de l'agent (le bloc 7 du prompt) — ce qu'il dit avoir fait, ce qu'il dit ne pas avoir fait.
2. Relire le diff (pas les fichiers entiers) avec la méthode 4.2 et les signatures 4.3.
3. Lancer la validation machine : `make test && make lint && make build` (ou vos commandes AGENTS.md). Ne faites jamais confiance au « j'ai lancé les tests » de l'agent : relancez vous-même.
4. Tester le parcours principal à la main : 5 minutes d'utilisation réelle de la fonctionnalité. Un agent ne teste pas « comme un humain » — il teste ce qu'il a codé, pas ce que l'utilisateur vit.
5. Regrouper les demandes de correction en une passe : une liste numérotée, fichier par fichier, avec référence à la définition de fini. Un seul aller-retour, pas un ping-pong.

4.6 Quand dire non (et comment)

Vous direz non quand : le périmètre est dépassé, les tests ont été affaiblis, un secret a été commité, ou le code est incompréhensible sans une réécriture massive. Le « non » professionnel :

> « Je ne merge pas ce changement : la modification de tests/paiement.test.js retire l'assertion sur le montant (signature n°3). Merci de restaurer l'assertion, d'ajouter le test du cas "paiement refusé", et de relancer la suite. »

Précis, référencé, actionnable. Pas de « c'est moche », pas de « ça va pas » — l'agent a besoin d'instructions exécutables, comme n'importe quel collaborateur junior motivé.

4.7 Le piège n°4 : la review-béquille

Ne délégez pas la review à un autre agent « pour gagner du temps » — pas encore. Au module 6, la révision croisée entre agents a sa place (avec vos critères dans le prompt de review), mais elle complète votre review, elle ne la remplace pas. Les premières missions, vous relisez tout. C'est ce qui construit votre œil.

L'exemple réel

Situation. Karim reçoit le diff de la mission PAIEMENT-STRIFE : 15 fichiers, 1 284 lignes, 3 créés. Le résumé de l'agent est propre. Karim applique la méthode.

1. `--stat` : il remarque `src/utls/format.ts` modifié — hors périmètre. Signature n°4.
2. Lecture du diff : il trouve une abstraction `PaiementServiceFactory` pour un seul usage — signature n°1. Il repère aussi `stripe.charges.create({...})` sans vérification d'erreur dans un chemin — risque réel.
3. `-- '/test'` : les 5 tests d'intégration sont là, mais l'un d'eux vérifie `expect(response.status).toBe(200)` sans vérifier le contenu — test creux (signature n°3, version douce).
4. Il relance `make test && make lint` : verts. Il teste à la main un paiement refusé : le serveur répond 500 au lieu du JSON d'erreur prévu.
5. Il regroupe : « 1) annule la modif de `src/utls/format.ts` (hors périmètre) ; 2) supprime la factory, un seul usage ; 3) gère l'erreur stripe dans le endpoint (réponse 500 sur paiement refusé, attendu : `{error: "...}"` avec code 402) ; 4) renforce le test : vérifier le montant et le statut dans la réponse. »

Un aller-retour. Le second diff est propre, Karim relance les tests, teste le parcours, merge. Total review : 25 minutes, dont 5 de test manuel.

L'exercice — Module 4

Objectif : aiguïser votre œil sur un diff d'agent.

1. Prenez le diff de votre mission du module 3 (ou d'une mission passée d'un collègue/agent).
2. Appliquez la méthode 4.2 : `--stat`, lecture ciblée, contrôle des fichiers de test, contrôle des dépendances.
3. Cherchez explicitement les 7 signatures (4.3) — notez-en au moins 3, même mineures.
4. Rédigez une passe de corrections groupée (4.5 étape 5) : liste numérotée, fichier par fichier.
5. Appliquez le workflow de validation complet (5 étapes) avant de merger.

Critères de réussite :

- ☐ Vous avez vérifié les fichiers hors périmètre AVANT de lire le code
- ☐ Vous avez relancé les tests vous-même (pas confiance dans le « déjà fait » de l'agent)
- ☐ Vous avez testé le parcours principal à la main
- ☐ Votre passe de corrections tient en une liste numérotée de moins de 10 items
- ☐ Vous n'avez pas utilisé « c'est moche » ou « ça va pas » — chaque point est actionnable

Module 5 — Le garde-fou tests

La leçon

5.1 Les tests comme contrat de délégation

Un agent de code est optimiste par construction : il produit du code qui « devrait marcher ». Le seul juge objectif — celui qui ne discute pas, ne flatte pas, ne « devrait » pas — c'est la suite de tests. Le module 5 consiste à faire des tests le contrat de chaque mission : avant de déléguer, vous définissez ce que « marcher » veut dire, et l'agent ne peut pas déclarer fini tant que le contrat n'est pas rempli.

5.2 Le test-first avec agent

La méthode la plus efficace pour piloter un agent :

1. Vous écrivez (ou faites écrire) les tests d'abord : les cas nominaux, les cas limites, les cas d'erreur. Ils échouent — c'est voulu, c'est le rouge.
2. L'agent implémente la fonctionnalité pour faire passer les tests — c'est le vert.
3. La mission est finie quand les tests passent — pas avant, pas après.

Pourquoi c'est si puissant : les tests sont la spécification exécutable de la mission. Au lieu de décrire le comportement en français (que l'agent interprète), vous le décrivez en tests (que l'agent ne peut pas interpréter : ça passe ou ça casse). Et la review devient triviale : si les tests sont bons et qu'ils passent, le comportement est là.

5.3 Le contrat de tests : à coller dans chaque prompt

Le template garde-fou-tests.md se colle tel quel dans le bloc « Définition de fini » de vos prompts de mission. Il contient les règles non négociables :

- Règle 1 — Jamais de suppression de test sans justification écrite dans le résumé. Un test supprimé = une garantie perdue.
- Règle 2 — Jamais de modification de test pour le faire passer. Si un test échoue, on corrige le code, pas le test. L'agent doit signaler tout test qu'il juge erroné, sans le modifier.
- Règle 3 — Tests d'abord, code ensuite (pour les nouvelles fonctionnalités) : l'agent écrit les tests qui échouent, montre le rouge, puis implémente jusqu'au vert.
- Règle 4 — Un test doit échouer si la fonctionnalité casse. C'est la définition d'un test utile : si vous pouvez retirer la fonctionnalité et que les tests passent toujours, les tests sont creux.
- Règle 5 — La suite complète doit passer : make test en entier, pas « les 3 tests que j'ai écrits ». Les tests existants comptent autant que les nouveaux.

5.4 La hiérarchie du filet

Par ordre de puissance (du plus grossier au plus fin) :

1. Types (tsc --noEmit, mypy) — attrapent les hallucinations d'API et les signatures erronées (signature n°2 du module 4).
2. Lint (eslint, ruff, golangci-lint) — attrapent le style, les imports morts, les anti-patterns évidents (signatures n°6 et n°7).
3. Tests unitaires — attrapent la logique par fonction.
4. Tests d'intégration — attrapent les interactions (route + base + service), c'est le niveau des parcours métier.

5. CI — rend le filet obligatoire : rien ne merge sans que tout passe, automatiquement, sur une machine propre.

Un agent piloté correctement subit ce filet à chaque étape : il type, il lint, il teste, et la CI le vérifie. Vous n'avez plus à faire confiance — vous vérifiez.

5.5 Détecter les tests creux

C'est la compétence d'expert du module. Trois cas classiques :

- L'assertion vide : `expect(foo).toBeDefined()` ou pire, `expect(true).toBe(true)` — le test « passe » sans rien vérifier. Détection : lire les assertions, pas seulement les noms de test.
- Le test qui teste l'implémentation : il vérifie une fonction interne plutôt que le comportement — il casse à chaque refactoring sans signaler de vraie régression.
- Le test qui passe sans la fonctionnalité : commentez/retirez la fonctionnalité, si les tests passent toujours, ils sont creux. C'est le test ultime — faites-le au moins une fois par mission pour étalonner votre confiance.

5.6 Le piège n°5 : le « j'ai lancé les tests » de l'agent

Les agents affirment volontiers « tous les tests passent » sans les avoir lancés, ou après avoir lancé uniquement les nouveaux tests. Règle absolue : la validation machine est relancée par vous (module 4, étape 3) et par la CI (5.4). Le « j'ai lancé les tests » de l'agent est une information, jamais une preuve.

L'exemple réel

Situation. Mégane (module 1) reçoit sa seconde mission : « calcul du total d'une commande avec remise et TVA ». Son tuteur lui impose le test-first.

Ce qu'elle fait.

1. Elle écrit d'abord `tests/total.test.js` : total simple, remise de 10 %, remise plafonnée, TVA à 20 %, cas sans article, cas montant négatif (doit lever une erreur). Six tests, tous rouges.
2. Elle rédige le prompt de mission avec le contrat de tests dans la définition de fini : « implémente `src/total.js` pour faire passer les 6 tests de `tests/total.test.js`. Règle 2 : ne modifie pas les tests. Règle 5 : la suite complète (make test) doit passer. »
3. L'agent implémente, les tests passent, elle relance make test elle-même : verts.
4. Le test ultime : elle commente la gestion de la remise dans `src/total.js` et relance le test de remise — il échoue. Le filet est réel.
5. Elle merge, et montre à son tuteur : les 6 tests écrits AVANT le code, le diff minimal, le filet vérifié. Elle a démontré exactement la compétence que le marché paie : la rigueur.

L'exercice — Module 5

Objectif : installer le garde-fou tests sur une mission.

1. Prenez une fonction simple de votre repo (ou écrivez-en une : `calculTVA(prix, taux)`).

2. Écrivez 5 tests d'abord : cas nominal, cas limite, cas d'erreur, cas de bord (zéro, négatif), cas « qui casse si on retire la fonctionnalité ».
3. Vérifiez qu'ils échouent (rouge) avant l'implémentation.
4. Déléguez l'implémentation à l'agent avec le contrat de tests (templates/garde-fou-tests.md) dans la définition de fini.
5. Vérifiez : suite complète verte, tests non modifiés, et test ultime (retirez la fonctionnalité → un test échoue).

Critères de réussite :

- ☐ Vos 5 tests étaient rouges avant l'implémentation
- ☐ L'agent n'a modifié aucun de vos tests
- ☐ Le test ultime échoue quand la fonctionnalité disparaît
- ☐ Vous avez relancé la suite complète vous-même, pas seulement les nouveaux tests

Module 6 — Monter une équipe d'agents

La leçon

6.1 Quand passer au multi-agents

Un seul agent bien piloté suffit pour la majorité des missions. Passez au multi-agents quand au moins deux de ces conditions sont réunies :

- La mission dépasse une journée de travail humain.
- Le travail se découpe naturellement en domaines distincts (API, front, tests, migration) qui ne se marchent pas dessus.
- Vous avez besoin de parallélisme : plusieurs fonctionnalités indépendantes à livrer.
- Vous voulez de la révision croisée : un agent relit le travail d'un autre.

En dessous : restez mono-agent. Le multi-agents a un coût de coordination réel (6.5) — il ne se justifie que par le volume.

6.2 Les rôles

Dans une équipe d'agents, vous êtes l'architecte et le chef de projet. Les rôles :

- Vous — le specifieur : vous écrivez les fiches de tâches, vous définissez les dépendances et l'ordre, vous validez les merges. Vous ne codez pas — vous décidez.
- L'implémenteur : reçoit une fiche de tâche, produit le diff qui fait passer ses tests.
- Le reviewer : reçoit le diff d'un implémenteur + la checklist de review (templates/review-checklist.md), produit une liste de corrections.
- Le testeur : écrit les tests d'abord (module 5) pour les fonctionnalités à venir.

Un même agent peut changer de rôle d'une mission à l'autre — ce qui compte, c'est que les rôles ne

soient pas mélangés dans une même exécution : l'agent qui implémente ne se relit pas lui-même.

6.3 Le contrat inter-agents : la spec est le seul canal

La règle d'or du multi-agents : les agents ne communiquent pas entre eux — ils communiquent par les specs. Pas de « passe-moi le fichier », pas de « dis-lui que j'ai changé l'API » : chaque agent ne connaît que sa fiche de tâche, le repo, et les conventions (AGENTS.md). Toute information partagée passe par les fiches et par git.

Concrètement :

- La fiche de tâche d'un agent référence les livrables des fiches précédentes (« l'endpoint GET /orders existe déjà, spec FICHE-03 »).
- Les conventions d'interface (contrats d'API, noms de fonctions partagées) sont écrites dans les fiches, pas découvertes.
- Si deux fiches dépendent l'une de l'autre, elles sont ordonnancées (6.4) — jamais exécutées en parallèle sur les mêmes fichiers.

6.4 L'ordonnancement : le graphe de mission

Avant de lancer la première mission, dessinez le graphe :

1. Découpez la mission en fiches de tâches indépendantes (chacune = une fiche spec-fiche-tache.md, un objectif mesurable, une définition de fini).
2. Identifiez les dépendances : qu'est-ce qui doit exister avant quoi ? (La migration DB avant l'implémenteur API ; l'API avant le front ; les tests d'abord partout.)
3. Sérialisez les fichiers partagés : si deux fiches touchent src/models/order.js, elles ne tournent pas en parallèle — ou l'une est réécrite pour ne pas le toucher.
4. Définissez les jalons : à chaque jalon (ex. « API verte + tests »), vous validez avant de lancer la vague suivante.

Un graphe typique pour « portail client » :

- Vague 1 (parallèle) : FICHE-01 migration DB (testeur écrit les tests de migration), FICHE-02 tests d'API (rouges).
- Vague 2 : FICHE-03 implémenteur API (fait passer les tests de FICHE-02).
- Vague 3 : FICHE-04 front (consomme l'API de FICHE-03), FICHE-05 review croisée (le reviewer vérifie le diff de FICHE-03).
- Vague 4 : FICHE-06 intégration finale + votre review complète.

6.5 La révision croisée

Pour les missions importantes, ajoutez un rôle reviewer : un agent relit le diff d'un implémenteur avec la checklist de review dans son prompt, et produit la liste de corrections. Le prompt de review suit les 7 blocs du module 3, avec :

- Rôle : « reviewer senior, tu ne modifies aucun fichier, tu produis une liste de corrections. »
- Objectif : « vérifier que le diff respecte la fiche FICHE-03 et la checklist templates/review-checklist.md. »

- Périmètre : « lecture seule. Interdit : éditer, committer, exécuter autre chose que les tests. »
- Format : « liste numérotée : fichier, problème, correction proposée. Max 15 items. »

La review croisée attrape les signatures que vous avez peut-être manquées, mais elle ne remplace pas votre validation finale (module 4) : vous restez le seul à merger.

6.6 Les coûts et limites du multi-agents

- Le contexte n'est pas partagé : chaque agent a sa fenêtre de contexte, sa lecture du repo. Deux agents peuvent « savoir » des choses contradictoires. D'où la règle : la spec est le seul canal.
- Les coûts s'additionnent : 4 agents = 4 contextes chargés. Une vague de missions multi-agents coûte 5 à 15 € de tokens — c'est le prix d'une heure de dev humain, mais surveillez les missions qui tournent en boucle.
- La dette de coordination : chaque fiche mal écrite se répercute sur tous les agents qui en dépendent. Le multi-agents amplifie la qualité de vos specs — et vos erreurs.
- Git est le juge de paix : deux agents sur le même fichier = conflits. La règle 6.4 (sérialisation) les évite ; quand ils arrivent, vous arbitrez sur le diff, pas dans un chat.

6.7 Le piège n°6 : l'usine à agents

Le fantasme : 10 agents qui codent en parallèle pendant que vous regardez. La réalité : 10 agents qui se marchent dessus, des conflits partout, des specs contradictoires, et vous qui passez plus de temps à arbitrer qu'à développer. Montez progressivement : 1 agent (modules 1-5), puis 2 (vous + un implémenteur + un reviewer en révision croisée), puis 3-4 quand le graphe de mission est rodé. La maturité se mesure au nombre de missions terminées sans incident, pas au nombre d'agents lancés.

L'exemple réel

Situation. Karim veut livrer un « portail client » complet : authentification, liste des commandes, export CSV, page de profil. Estimation humaine : 4-5 jours. Il monte sa première équipe de 3 agents.

Ce qu'il fait.

1. Découpage : 6 fiches — F1 migration + modèle (avec tests d'abord), F2 auth (tests d'abord), F3 API commandes + CSV, F4 front liste, F5 front profil, F6 review croisée + intégration.
2. Graphe : vague 1 = F1 + F2 (parallèle, fichiers disjoints). Vague 2 = F3 (dépend de F1). Vague 3 = F4 + F5 (dépendent de F2/F3). Vague 4 = F6.
3. Lancement : 2 agents en parallèle sur F1 et F2, budgets de 30 min, approbation par étape. Il valide chaque vague avant la suivante.
4. Revue croisée : en vague 3, l'agent F5 relit le diff de F4 (prompt review en lecture seule, checklist). Il trouve 3 items — dont un oubli de gestion d'erreur sur l'API que Karim aurait mis 20 minutes à voir.
5. Validation finale : Karim applique le workflow du module 4 sur chaque merge, lance la suite complète, teste les parcours à la main.

Résultat. 4 jours de travail humain livrés en 2 jours calendaires (dont 1,5 jour de Karim en pilotage et review). Coût tokens : ~11 €. Le portail est livré avec 34 tests verts, chaque merge validé, aucun conflit

— parce que le graphe et les specs étaient bons avant le premier agent lancé.

L'exercice — Module 6

Objectif : préparer une mission multi-agents sans la lancer.

1. Prenez une mission de plus d'une journée (votre propre projet).
2. Découpez-la en 4 à 6 fiches de tâches indépendantes (templates/spec-fiche-tache.md).
3. Dessinez le graphe : vagues, dépendances, jalons, fichiers partagés à sérialiser.
4. Écrivez le prompt de review croisée (lecture seule, checklist).
5. Simulez le lancement : pour chaque fiche, vérifiez que l'objectif est mesurable et la définition de fini vérifiable sans ambiguïté.

Critères de réussite :

- ☐ Aucun fichier partagé entre deux fiches de la même vague (ou sérialisation prévue)
- ☐ Chaque fiche a une définition de fini vérifiable (tests, commandes)
- ☐ L'ordre des vagues respecte les dépendances
- ☐ Votre prompt de review interdit explicitement l'édition de fichiers
- ☐ Vous savez nommer le coût estimé de la vague complète en tokens

Checklist finale

Faites le point après la lecture et votre première mission. Cochez ce qui est acquis ; pour le reste, retournez au module indiqué.

Socle (modules 1-2)

- ☐ J'ai choisi un agent sur les 5 critères (contrôlabilité, transparence, coût, contexte, écosystème) — pas sur la hype
- ☐ Mon repo a un AGENTS.md lu par l'agent (vérifié par un test de résumé)
- ☐ Mon Makefile expose install / test / lint / build / dev
- ☐ main est protégée, chaque mission a sa branche feature/
- ☐ Mon agent est configuré : approbation par étape, budget de temps, whitelist de commandes, pas de push

Mission (modules 3-5)

- ☐ Je rédige une fiche de tâche avant chaque prompt de mission
- ☐ Mes prompts ont les 7 blocs, avec un objectif mesurable et un périmètre interdit
- ☐ Ma définition de fini contient au moins 4 conditions vérifiables
- ☐ Je relance moi-même les tests, le lint et le build (jamais confiance au « déjà fait »)
- ☐ Je lis le diff avec la méthode --stat / ciblée / tests / dépendances
- ☐ Je repère les 7 signatures du code d'agent sans aide
- ☐ Mes missions imposent le contrat de tests (pas de suppression, pas de test modifié pour passer)
- ☐ Je teste le parcours principal à la main avant de merger
- ☐ Je regroupe mes demandes de correction en une seule passe numérotée

Équipe (module 6)

- [] Je découpe les grosses missions en fiches indépendantes ordonnancées
- [] La spec est mon seul canal de communication entre agents
- [] Je fais relire les diffs importants par un agent reviewer en lecture seule
- [] Je reste le seul à merger

Résultat mesurable

- [] J'ai délégué une mission complète (fiche → prompt → review → merge) sans perdre le contrôle
- [] Je connais le coût token réel de mes 3 dernières missions
- [] Je suis capable de relire un diff d'agent en moins de 15 minutes

Glossaire

| Terme | Définition |

|---|---|

| Agent de code | Programme qui planifie et exécute des tâches de développement (lire, éditer, tester, commit) en s'appuyant sur un grand modèle de langage. |

| Harness | Infrastructure qui encadre l'agent : accès aux fichiers, exécution de commandes, journalisation, approbations. Les projets open source comme deepseek-harness ou CodexPlusPlus en sont des exemples. |

| CLI | Command Line Interface — agent piloté depuis le terminal (Claude Code, Codex CLI, Aider...). |

| Fenêtre de contexte | Quantité maximale de tokens que le modèle peut « voir » en une session (fichiers lus, historique, instructions). Détermine la taille de code qu'un agent peut manipuler. |

| Token | Unité de découpage du texte pour un modèle (~3/4 de mot en anglais, variable en français). L'unité de facturation des API. |

| Contexte (coût) | Les tokens envoyés au modèle (fichiers, instructions, historique). Souvent 10 à 20 fois le volume de la sortie. |

| Sortie | Les tokens produits par le modèle (code, réponses). Facturée plus cher que le contexte. |

| MCP | Model Context Protocol — standard ouvert pour brancher des outils externes (base de données, ticketing, navigateur) sur un agent. |

| AGENTS.md | Fichier de contexte à la racine d'un repo, lu automatiquement par les agents : stack, commandes, conventions, pièges, règles. |

| Definition of Done (DoD) | Liste des conditions vérifiables pour qu'une mission soit terminée (tests verts, lint, types, migration, docs). |

| Périmètre | Zones autorisées et interdites d'une mission (fichiers, commandes). Première ligne de défense contre l'égarement. |

| Spec | Spécification écrite d'une mission (fiche de tâche). Dans le multi-agents, unique canal de communication entre agents. |

| Diff | Différence entre deux états du code (avant/après une mission). La matière première de la review. |

- | Review | Lecture critique des changements proposés avant merge. Jamais automatique. |
- | Test-first | Écrire les tests avant l'implémentation ; l'implémentation est finie quand les tests passent. |
- | Test creux | Test qui « passe » sans vérifier le comportement (assertion vide, test qui teste l'implémentation, test qui passe sans la fonctionnalité). |
- | CI | Continuous Integration — exécution automatique des vérifications (types, lint, tests) sur une machine propre à chaque proposition de merge. |
- | Merge | Intégration d'une branche dans une autre. Dans cette méthode : un acte de validation humaine. |
- | Conventional commits | Convention de messages de commit (feat:, fix:, test:, refactor:) qui rend l'historique lisible. |
- | Branche | Copie de travail isolée du code. Une branche par mission, main protégée. |
- | Whitelist / blacklist | Liste des commandes autorisées / interdites pour l'agent. |
- | Révision croisée | Un agent relit le diff d'un autre agent (en lecture seule) avec une checklist. Complète, ne remplace pas, votre review. |
- | Graphe de mission | Représentation des fiches de tâches, de leurs dépendances et de leur ordre d'exécution. |
- | Rouge / vert | État des tests : rouges (ils échouent), verts (ils passent). La séquence test-first est rouge → vert. |

Annexe — Les 5 templates

Le pack complet inclut 5 templates prêts à l'emploi dans le dossier templates/. Ils accompagnent les modules :

1. prompt-mission.md (module 3) — le squelette du prompt en 7 blocs, avec un exemple complet rempli. Copiez, adaptez, envoyez.
2. spec-fiche-tache.md (modules 3 et 6) — la fiche de réflexion avant le prompt : objectif, périmètre, contraintes, DoD, dépendances. Le point de départ de toute mission, mono ou multi-agents.
3. review-checklist.md (module 4) — la checklist de review : périmètre, signatures du code d'agent, sécurité, tests, validation machine. À utiliser pour vos reviews ET pour le prompt de révision croisée.
4. garde-fou-tests.md (module 5) — le contrat de tests à coller dans la définition de fin de chaque prompt, plus la grille des cas à couvrir.
5. retrospective.md (modules 4-6) — le bilan après chaque mission : ce qui a marché, ce qui a coïncé, le coût réel, les règles à ajouter à votre AGENTS.md. C'est la boucle d'amélioration qui transforme la pratique en expertise.

Formation « Pilotez vos agents de code » — édition 2026. Les commandes et tarifs d'API cités sont ceux en vigueur à la date de rédaction ; les ordres de grandeur (coût par mission, volume de tokens) restent stables d'un fournisseur à l'autre. Vérifiez toujours la tarification de votre fournisseur avant de lancer une mission.